TX

Turn을 복구 가능한 거래로 닫는 법

stream, tool loop, fallback, terminal, transcript, resume와 projection 정합성

> **한 turn은 원자적이지 않지만, 모든 부분 사건을 일관된 protocol과 durable history로 합류시켜야 한다.**

대상
v2.1.88 / 2ca5ddabfed5

관점
state machine · recovery · durability

제3자 복원 소스에 대한 교육용 분석이다. 누락된 모듈과 복원 한계를 구현 사실로 보완하지 않는다.

turn은 transaction처럼 다뤄지지만 ACID transaction은 아니다

모델 응답, tool effect, UI projection, transcript write가 단계적으로 발생한다. 중간 실패 시 모두 rollback할 수 없으므로 “무엇이 발생했는가”를 일관되게 남기는 것이 핵심이다.

```
Turn = state transition + external effects + reconciliation record
```

이 시스템의 성공은 마지막 assistant text만으로 정의되지 않는다. tool 결과가 다음 request에 합류하고, terminal reason이 caller에 전달되며, process가 죽어도 resume가 이미 수락된 사용자 입력과 발생한 effect를 복원해야 한다.

query loop는 암묵적 재귀를 명시적 State로 바꾼다

메시지, tool context, compact tracking, recovery count, pending summary, stop-hook, turn count, transition reason이 반복 사이를 이동한다.

state	수명	왜 명시적인가
messages	loop 전 구간	다음 request와 resume의 입력
toolUseContext	tool 실행 후 갱신	permission/read state/app state 전달
compact tracking	여러 iteration	threshold·recompaction·budget 의미
recovery guards	오류 반복 동안	동일 compact/retry 무한 루프 방지
pending summary/prefetch	비동기 overlap	critical path 밖의 결과를 이후 합류
transition	iteration 경계	왜 continue했는지 테스트/관측 가능

코드 좌표 src/query.ts

async generator는 상태 머신의 외부 관찰 계약이다

generator가 내부 State를 대신하지 않는다. 내부 transition을 수행하면서 event를 즉시 내보내고, 종료 시에는 Terminal 값을 반환하는 인터페이스다.

핵심 시간 구조

```
async function* query(): AsyncGenerator<Event, Terminal> {
  while (true) {
    for await (const event of modelStream) yield event
    for await (const result of runTools()) yield result
    if (done) return { reason: "completed" }
    state = next(state)
  }
}
```

`yield`는 UI/SDK에 관찰을 제공하면서 continuation과 local frame을 보존한다. `return`은 terminal reason을 iterator completion value로 구분한다. 소비자가 `.return()`하거나 throw가 발생하면 wrapper의 완료 코드가 실행되지 않는 비대칭도 설계에 포함된다.

코드 좌표 src/query.ts · ECMAScript AsyncGenerator Objects

yield된 event와 내부 정식 상태는 같지 않다

stream event를 보여 줬다고 conversation state에 commit된 것은 아니며, 내부 배열에 넣었다고 모든 consumer에게 노출되는 것도 아니다.

사건	외부 projection	내부 처리
text/thinking delta	즉시 UI/SDK stream 가능	assistant message에 누적
tool_use block	호출 표시	needsFollowUp와 executor에 등록
progress	spinner/tool 상태	transcript 정책에 따라 기록
tombstone	기존 partial row 제거	failed attempt arrays 초기화
compact boundary	SDK/session 경계	이전 in-memory messages release
terminal	result/status	generator return으로 caller settlement

핵심 구분 event는 “무슨 일이 관찰됐는가”, state는 “다음 결정이 무엇을 진실로 사용할 것인가”를 답한다.

streaming은 UI 최적화보다 protocol 조립 문제다

raw provider event에서 content block, usage, stop reason, tool input JSON을 누적해 유효한 assistant message를 만든다.

01	request	cache-aware system/tools/messages와 AbortSignal 생성
02	message_start	usage와 request identity 초기화
03	content deltas	text/thinking/tool input을 incremental accumulate
04	content stop	완성된 block을 assistant message로 projection
05	message_delta	usage와 실제 stop_reason 보정
06	message_stop	cost·timing·stream resource 정리

raw stream을 사용하는 이유로 $O(n^2)$ partial JSON parsing 회피가 주석에 명시돼 있다. 즉 transport 선택은 renderer 취향이 아니라 장문 tool input의 성능과 memory에 직접 연결된다.

코드 좌표 src/services/api/claude.ts

tool_use/result pairing은 가장 중요한 protocol 불변조건이다

모델이 tool_use를 내보낸 뒤 stream·tool·process가 실패해도 다음 API history에는 같은 id의 tool_result가 필요하다.

pairing invariant

```
assistant: tool_use(id=A)
user:      tool_result(tool_use_id=A)

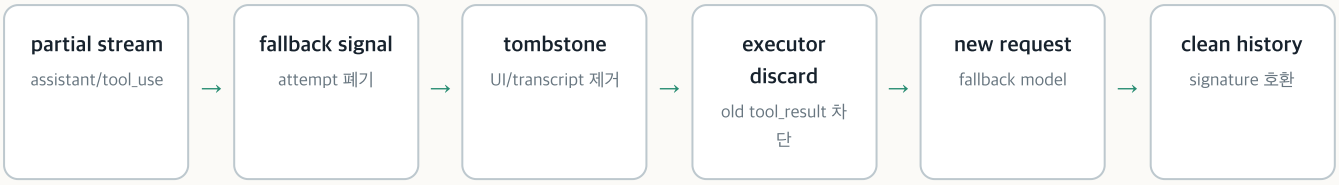
abort/error → synthetic error result for A
fallback    → tombstone old attempt; never leak result A into new ids
```

이 규칙 때문에 abort 처리보다 먼저 remaining executor를 drain하고, exception path에서 missing result를 생성한다. 사용자에게 “취소됨”을 보여 주는 것보다 provider protocol을 유효하게 유지하는 것이 우선이다.

코드 좌표 src/query.ts · src/query.ts

fallback은 재호출이 아니라 failed attempt 제거를 포함한다

streaming attempt가 일부 assistant/tool event를 이미 내보낸 뒤 fallback되면 이전 projection과 executor를 정리해야 한다.



특히 protected thinking signature는 model-bound일 수 있어 fallback 전 제거한다. “같은 prompt를 다른 모델에 재전송”하는 단순화가 protocol·cache·effect 관점에서 틀린 이유다.

코드 좌표 src/query.ts · src/query.ts

회복 가능한 오류는 외부에 바로 commit하지 않는다

prompt-too-long, media size, max-output 같은 오류를 즉시 SDK/UI에 내보내면 consumer가 turn을 종료할 수 있다. runtime은 recovery 여부가 결정될 때까지 보류한다.

오류	가능한 회복	guard
prompt too long	collapse drain/reactive compact	한 번 시도 후 반복 방지
max output tokens	limit 조정·continuation	회복 횟수 제한
media size	resize/remove/retry	gate를 turn 시작에 latch
stream stall	watchdog abort + non-stream fallback	timer cleanup·double effect 고려

> **transaction 관점**
>
> withhold는 provisional event다. 성공한 회복 뒤에는 원래 오류를 commit하지 않고, 회복 실패 시에만 최종 failure projection을 만든다.

코드 좌표 src/query.ts · src/services/api/claude.ts

종료는 boolean이 아니라 caller가 해석할 이유다

completed와 실패·중단은 UI 메시지뿐 아니라 SDK result, lifecycle notification, cleanup을 다르게 만든다.

reason	의미	caller 책임
completed	tool follow-up와 stop hook까지 수렴	result·usage·completion 처리
blocking_limit	context 정책이 호출 차단	compact 안내/재시도 선택
model_error/image_error	request/runtime 실패	진짜 오류를 보존
aborted_tools	실행 중 사용자/상위 취소	부분 효과와 interruption 표시
hook_stopped/stop_hook_prevented	정책이 continuation 중단	정책 이유와 상태 유지
max_turns	운영 한도	현재 conversation을 재개 가능하게 정리

복원 한계

`query/transitions.ts` 선언 파일은 복원본에서 누락됐다. variant 명칭은 query의 return 지점과 QueryEngine 처리에서 확인한다.

“모델이 끝났다”와 “turn이 끝났다”는 다르다

tool_use가 없더라도 stop hook이 continuation을 막거나 새 blocking message로 재시도를 요구할 수 있다.

stop hook blocking 뒤 reactive compact guard를 잘못 초기화하면 compact→여전히 too long→hook block→compact가 반복될 수 있다. 코드 주석은 실제로 수천 API call을 태울 수 있는 무한 루프를 설명한다.

개발 노력의 핵심	policy extension을 loop에 넣을 때는 “다시 실행할 수 있다”가 아니라, 재시도 fingerprint와 monotonic guard가 있는지를 확인해야 한다.

코드 좌표 src/query.ts

새 사용자 입력은 현재 turn과 경쟁한다

tool 실행 중 들어온 prompt·task notification·slash command를 모두 같은 방식으로 처리할 수 없다.

입력	처리	이유
prompt	현재 tool 후 attachment 또는 다음 turn queue	사용자 의도를 잃지 않음
task notification	수신 agent scope에 따라 drain	process-global queue의 오배송 방지
slash command	turn 종료 뒤 local processor	모델 text로 잘못 전달 방지
submit-interrupt	현재 turn abort 후 새 입력으로 맥락 제공	불필요한 interruption message 생략

queue entry는 lifecycle started/completed를 가지며, generator throw나 return에서는 completed가 의도적으로 누락될 수 있다. 이 비대칭은 failure 관측 신호다.

코드 좌표 src/query.ts · src/query.ts

transcript는 로그 파일이 아니라 재실행 상태다

사용자 입력을 model call 전에 기록하는 이유는 응답 전 process kill에도 session을 resume하기 위해서다.

interactive mode에서는 이 write를 await해 내구성을 우선하고, bare/script mode에서는 critical-path latency를 위해 fire-and-forget을 선택한다. 동일 기능에서도 사용 시나리오에 따라 durability/latency 정책이 다르다.

시점	정책	trade-off
user input accepted	응답 전 record	resume 보장 vs 수 ms latency
assistant block	ordered queue에 async enqueue	stream blocking 방지
terminal/desktop handoff	필요 환경에서 flush	process kill 전 bytes 보장
tombstone	orphan message 제거 기록	failed attempt 재등장 방지

코드 좌표 src/QueryEngine.ts · src/QueryEngine.ts

append-only는 단순하지만 resume projection은 어렵다

rewind, compact, fork, subagent, metadata update가 append-only JSONL 위에 branch와 대체 의미를 만든다.

세션 journal의 개념 모델

```
append log:
  message(uuid, parentUuid)
  compact_boundary(preserved segment)
  queue operation / metadata / tombstone

resume:
  parse → select chain → repair siblings/pairs
       → apply compact/snip projections → materialize messages
```

sessionStorage의 거대한 크기는 파일 append 자체보다 branch 선택, orphan 회복, stale metadata, compact relink, 대형 파일 memory 사용을 다루는 비용을 반영한다.

코드 좌표 src/utils/sessionStorage.ts · src/utils/sessionStorage.ts

in-memory 압축과 durable history는 같은 의미를 재생해야 한다

turn 중 context를 줄였는데 resume가 pre-compact history를 되살리면 token 폭발과 즉시 재압축이 발생한다.

문제	원인	필요한 record
pre-compact 부활	boundary가 preserved chain과 연결되지 않음	tail/head relink metadata
stale token usage	디스크 usage가 압축 전 값	boundary 이후 usage reset/projection
snip 반복	marker가 replay에서 제거되지 않음	snip decision과 filtered history
orphan tool result	single-parent walk가 sibling branch 누락	pair-aware recovery

검증 단위

"compact가 성공했다"가 아니라, 같은 session을 즉시 재개했을 때 model-visible history와 token state가 의미상 동일한지를 본다.

QueryEngine은 conversation ownership을 headless core로 옮기는 중이다

class는 messages, read cache, usage, abort, denial을 conversation lifetime으로 소유하고 submitMessage마다 turn을 연다.

코드 주석은 기존 `ask()`의 core logic을 standalone class로 추출했고 현재 headless/SDK가 사용하며 REPL은 future phase라고 명시한다. 이는 완성된 통합이 아니라 진행 중인 architectural seam이다.

소유

conversation state

mutableMessages·read state·usage가 turn 사이에 지속.

입력

submitMessage

prompt normalize와 local command까지 처리.

출력

SDK projection

core message를 SDKMessage/result로 변환.

제약

REPL 미통합

interactive lifecycle과 중복이 아직 존재.

코드 좌표 src/QueryEngine.ts · src/QueryEngine.ts

REPL은 단순 view보다 많은 lifecycle을 소유한다

현재 REPL의 onQueryImpl은 prompt context 조립, query 호출, stream event 반영, telemetry, turn completion까지 담당한다.

따라서 React/Ink를 renderer로만 설명하면 부족하다. interactive store의 최신 MCP/tool 상태, queryGuard, queued input, cancel 후 prompt restore 같은 session coordination이 component 안에 있다.

React/Ink 역할	core와 맞닿는 지점
input event	prompt/selection/queue/abort intent 생성
state subscription	stream/progress/message projection
render/reconcile	terminal buffer로 증분 출력
query callback	현재는 context와 lifecycle orchestration도 포함

코드 좌표 src/screens/REPL.tsx · src/components/PromptInput/PromptInput.tsx

generation guard가 cancel/resubmit race를 막는다

boolean isLoading을 check한 뒤 set하면 두 submit이 동시에 통과할 수 있다. queryGuard.tryStart는 확인과 상태 전이를 원자적으로 묶고 generation을 반환한다.

stale finally를 막는 generation token

```
g = guard.tryStart()
if (g == null) enqueue(newPrompt)
try { await runTurn() }
finally {
  if (guard.end(g)) settleThisGeneration()
}
```

cancel+resubmit에서는 이전 turn의 finally가 microtask로 늦게 실행될 수 있다. generation이 없으면 새 turn의 loading/controller 상태를 이전 turn이 지워 버린다.

코드 좌표 src/screens/REPL.tsx

React/Ink는 event projection의 구현 선택이다

영상의 설명처럼 React가 상태 변화에서 terminal UI를 만든다. 그러나 핵심 학습점은 renderer보다 어떤 상태를 static scrollback과 dynamic region으로 나누고, 얼마나 자주 projection을 갱신하는가다.

React

declarative projection

message/progress/input state를 component tree로 표현.

reconciler

host operations

React mutation을 Ink node tree에 적용.

Yoga

layout

terminal cell 제약으로 크기와 위치 계산.

ANSI renderer

buffer diff

바뀐 영역을 stdout에 쓰고 scrollback을 보존.

성능 해석

"항상 16ms로 batch한다"는 식의 고정 주기는 소스/측정 없이 보장할 수 없다. chunk rate, render count, bytes written, input latency를 함께 계측해야 한다.

코드 좌표 src/ink/reconciler.ts · src/ink/render-to-screen.ts

checkpoint는 성능 장식이 아니라 causal map이다

한 turn은 model latency만으로 설명되지 않는다. context load, schema build, stream, tool, hook, compact, transcript flush가 critical path를 나눈다.

측정	알 수 있는 것	오해하기 쉬운 것
TTFT	첫 model event까지	prompt/context 준비 시간과 분리 필요
OTPS	active stream 동안 출력 속도	tool/compact 대기 포함 turn 속도와 다름
tool duration	외부 effect 비용	permission dialog 시간과 분리 여부
hook duration	extension overhead	blocking/async hook 의미 차이
turn duration	사용자 체감 전체	background/subagent 수명과 다름
resume delta	in-session vs restored context 차이	단순 message count만으로 의미 동등성 보장 못함

코드 좌표 src/utils/queryProfiler.ts · src/screens/REPL.tsx

turn engine을 event protocol과 policy로 분리한다

REPL/SDK 통합의 목표는 component를 작게 만드는 것이 아니라 동일 turn semantics를 한 owner가 갖게 하는 것이다.

목표 구조의 최소 골격

```
ConversationEngine
  submit(InputIR) -> AsyncIterable<CoreEvent> + Terminal

policies:
  ContextPolicy | RecoveryPolicy | ToolSchedulePolicy

adapters:
  ReplProjection | SdkProjection | TranscriptJournal
```

- CoreEvent는 React node나 SDK wire type을 포함하지 않는다.
- Terminal과 recovery transition을 discriminated union으로 닫는다.
- journal write 위치를 engine lifecycle에 명시하고 durability mode를 입력으로 둔다.
- REPL migration 전 golden trace를 만들어 event/order/terminal을 비교한다.

> **가장 큰 위험**
>
> interactive path의 latency·cancel·prompt restore와 SDK path의 replay/flush가 서로 다른 암묵 계약을 갖고 있다. 공통분모만 추출하면 양쪽 의미를 잃을 수 있다.

복구 가능한 turn의 조건

모든 성공과 실패가 하나의 truth로 합류할 때만 다음 turn과 resume가 올바르다.

- yield된 partial event와 committed conversation state를 구분한다.
- 모든 tool_use/result id pairing을 abort·error·fallback에서도 보존한다.
- 회복 가능한 오류는 provisional로 보류하고 반복 guard를 둔다.
- terminal reason을 caller가 해석할 값으로 반환한다.
- 사용자 입력과 외부 effect를 process crash 전에 필요한 수준으로 journal에 남긴다.
- REPL·SDK·resume가 같은 core 사건을 의미상 동등하게 투영하는지 trace로 검증한다.